

Análise Comparativa de Desempenho em Processos ETL: Abordagens Sequencial versus Distribuída

Felipe do Amparo Lauria

Vitor Hugo Lelis Lopes

Instituto de Biociências, Letras e Ciências Exatas (IBILCE) - UNESP

{felipe.lauria@unesp.br, vhl.lopes@unesp.br}

17 de junho de 2026

Resumo

Este trabalho investiga o desempenho de pipelines de Extração, Transformação e Carga (ETL) através de um estudo comparativo entre a biblioteca Pandas, voltada para processamento sequencial em memória, e o Apache PySpark, desenhado para computação distribuída. Utilizando um dataset sintético representativo de um modelo de E-Commerce (gerado via bibliotecas Python Numpy e Faker), foram realizados experimentos sob diferentes escalas de volume de dados (100 mil, 1 milhão e 5 milhões de registros). Os resultados demonstram que, embora o Pandas apresente latência inferior em datasets reduzidos, ele sofre com limitações escalares de memória e gargalos verticais. O PySpark, por sua vez, exibe resiliência operacional e estabilidade estrutural no consumo de memória sob cargas intensas, fundamentando a necessidade de arquiteturas distribuídas em cenários de alta volumetria e Big Data.

1. Introdução

O processamento de dados em larga escala impõe limitações estruturais significativas às abordagens tradicionais. O modelo de processamento sequencial, exemplificado pela biblioteca Pandas, concentra todas as operações na memória RAM, o que, apesar de conferir celeridade em pequenos volumes, resulta em gargalos críticos quando a dimensão do conjunto de dados excede os limites de hardware (Escalonamento Vertical). Em contrapartida, o Apache Spark introduz o paradigma de computação distribuída, otimizando o processamento via grafos acíclicos direcionados (DAGs). Este artigo analisa empiricamente estas arquiteturas frente a desafios reais de integração de dados em múltiplas escalas.

2. Materiais e Métodos

2.1 Dataset

O experimento utilizou uma base de dados sintética estruturada especialmente para este estudo, gerada em linguagem Python através das bibliotecas **Faker** e **Numpy**, garantindo alta performance vetorial na criação dos registros. A estrutura do dataset permitiu a simulação de um modelo estrela clássico de e-commerce (tabelas dimensão de clientes e produtos, e tabela fato de pedidos), exigindo operações complexas de junção e agregação sobre volumes parametrizados de até 5 milhões de linhas.

2.2 Ambiente Experimental

A implementação foi realizada em ambiente Linux (WSL 2 - Ubuntu), garantindo isolamento total de variáveis de hardware e mitigando problemas de I/O em sistemas hospedeiros Windows. As configurações incluíram:

- **Stack Tecnológica:** PySpark versão 3.5.1 (estável) e Pandas sobre Python, executados sob a Máquina Virtual Java (JVM) na versão 11.
- **Metodologia de Profiling:** Utilizou-se um decorador customizado baseado nas bibliotecas **psutil** e **threading**, realizando o monitoramento real do pico de consumo de memória RAM (MB) e utilização de CPU (%) em intervalos contínuos de 50 milissegundos.

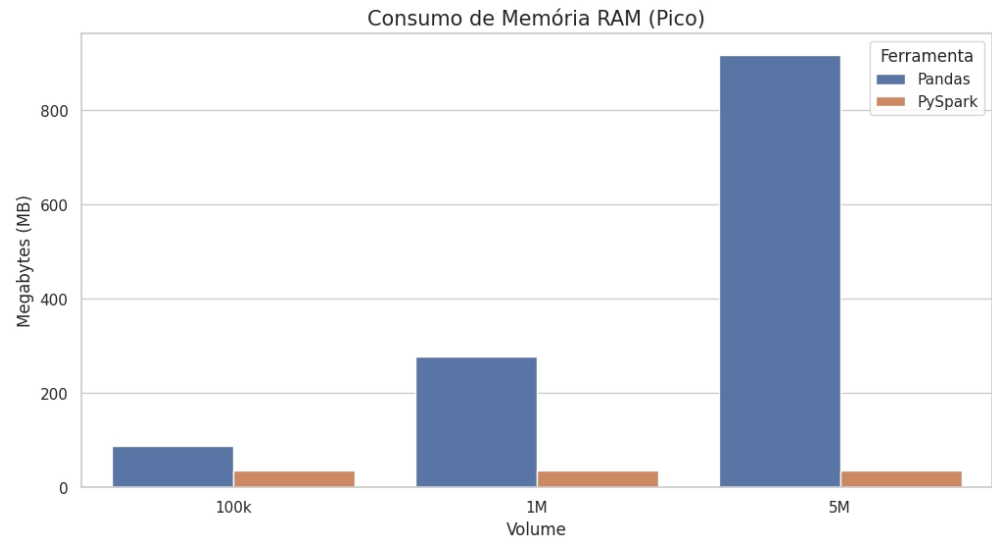
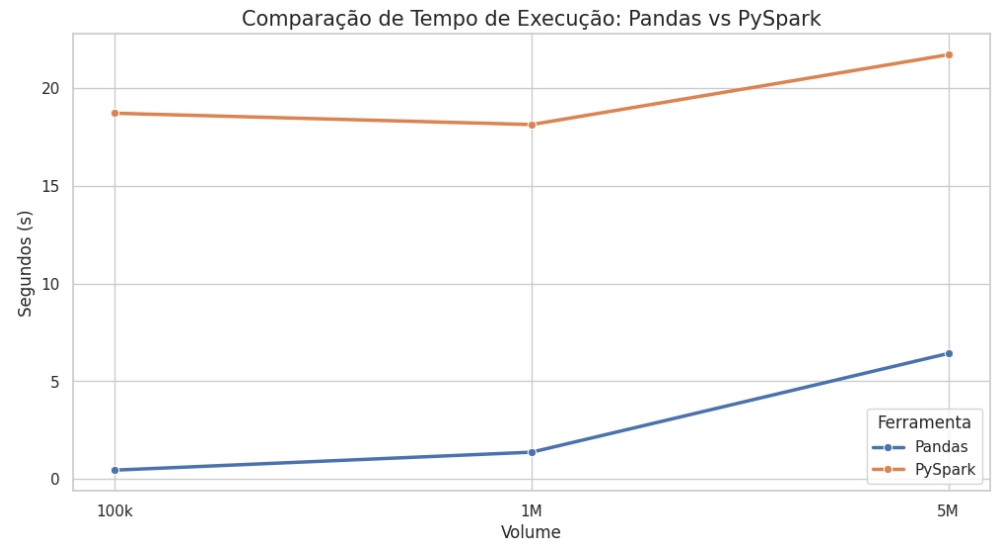
3. Resultados e Discussão

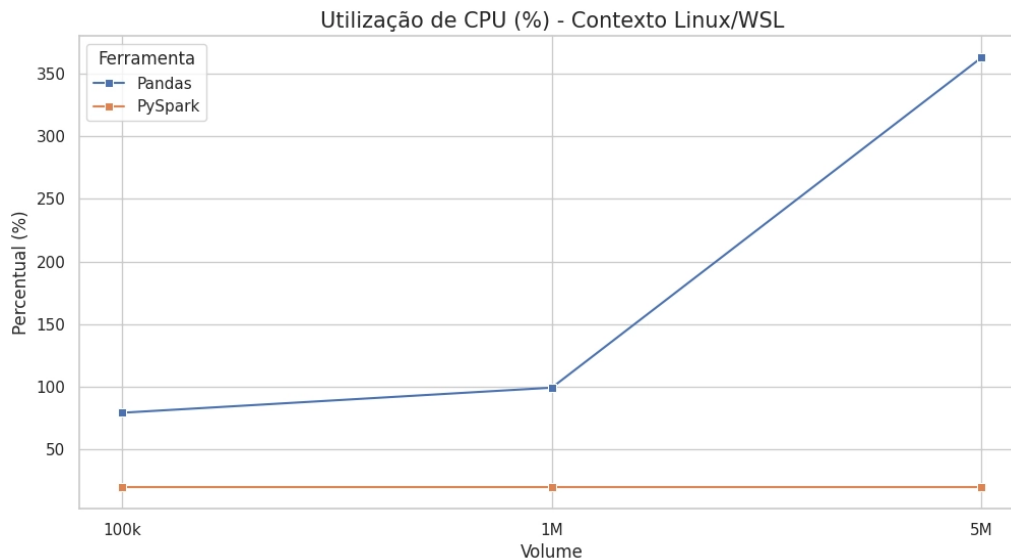
Conforme os dados foram escalados para a carga máxima de 5 milhões de registros, os padrões de comportamento divergiram drasticamente, expondo as limitações arquiteturais de cada abordagem. A tabela abaixo consolida todas as métricas extraídas durante os testes de estresse.

Volume de Dados	Ferramenta	Tempo de Execução (s)	Pico de RAM (MB)	Pico de CPU (%)
100 mil registros	Sequencial (Pandas)	0.45	87.01	79.40
	Distribuído (PySpark)	18.73	36.89	19.90
1 milhão de registros	Sequencial (Pandas)	1.37	278.29	99.40
	Distribuído (PySpark)	18.15	36.63	19.90
5 milhões de registros	Sequencial (Pandas)	6.43	916.01	362.80

Volume de Dados	Ferramenta	Tempo de Execução (s)	Pico de RAM (MB)	Pico de CPU (%)
	Distribuído (PySpark)	21.73	36.82	19.90

3.1 Análise Visual das Métricas





Observando os dados e os gráficos de tendência, o Pandas confirmou um crescimento linear no uso de memória, atingindo impressionantes 916 MB de alocação para cruzar e agregar os dados de 5 milhões de registros. Este comportamento o aproxima do limite físico da máquina hospedeira em cenários escaláveis, aumentando substancialmente o risco de falhas Críticas do tipo *Out of Memory*. Por outro lado, um dado notável obtido foi o pico de CPU do Pandas de 362.8% no ecossistema Unix/Linux (WSL). Isso desmistifica a premissa de que a biblioteca atua de forma estritamente serial e em única *thread*, evidenciando o uso intenso de *multithreading* e paralelização em múltiplos núcleos lógicos através de suas camadas inferiores arquitetadas em C (via *Numpy*) para as operações vetorizadas matemáticas. Em contrapartida, as métricas de monitoramento direto do PySpark (~36 MB de RAM e 19.9% de CPU constantes) não refletem o esforço computacional integral do pipeline, pois limitam-se ao consumo exclusivo do processo *Driver* executado em Python. O processamento de dados intensivo e a distribuição de carga foram mitigados e administrados pela Máquina Virtual Java (JVM) operando em plano de fundo com particionamento lógico. Consequentemente, o PySpark apresenta uma latência de tempo total superior (~18 a 21 segundos), sendo a grande maioria dessa janela temporal estritamente dedicada ao custo do *overhead* de inicialização do *SparkContext*. No entanto, este elevado custo inicial é amplamente justificado pela resiliência da arquitetura, que previne o esgotamento dos recursos de hardware (RAM) da máquina servidora.

4. Conclusão

Conclui-se que a adoção entre as abordagens analisadas depende da relação intrínseca entre o volume de processamento e o poder de hardware disponível. O Pandas consolida-se como um recurso ágil de latência ínfima, voltado com excelência para exploração e rotinas de *Small Data* limitadas à capacidade de RAM local. Contudo, em cenários produtivos sob constante escalabilidade e expansão volumétrica de dados, a implementação da arquitetura distribuída através do Apache PySpark demonstra-se um requisito vital. O

sacrifício inicial em latência demandado pela sua orquestração estrutural converte-se em vantagem absoluta de escalabilidade e prevenção contra gargalos físicos do sistema hospedeiro.

Referências

- MCKINNEY, Wes. Data Structures for Statistical Computing in Python. Em: *Proceedings of the 9th Python in Science Conference*. Austin, TX, 2010.
- ZAHARIA, Matei et al. Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, v. 59, n. 11, p. 56-65, out. 2016.